

Relatório sobre o 2º trabalho prático de EDA 2

Palm Island Neighbours



Universidade: Colégio Luís António Verney (Universidade de Évora)

Curso: Engenharia Informática

Identificações: Grupo g231

(Diogo Tavares Nº 58049 ,Francisco Rodrigues Nº 59119)

Ano: 2024/25

Objetivo do Trabalho

O objetivo deste trabalho é determinar o diâmetro de uma árvore que representa a topologia da Palm Island, ou seja, o maior caminho mínimo entre dois habitantes da ilha.

Em uma árvore (grafo acíclico e conexo), o diâmetro corresponde ao número máximo de conexões que devem ser percorridas para ir de um habitante a outro, seguindo o único caminho existente entre eles.

Para isso, o algoritmo implementado utiliza duas execuções consecutivas do Percurso em Largura (BFS).

A primeira BFS identifica o nó mais distante de um ponto inicial arbitrário, neste caso o habitante 1.

A segunda BFS, partindo desse nó distante, encontra o nó mais afastado dele, cuja distância entre os dois corresponde ao diâmetro da árvore. Esse método é eficiente e adequado para a restrição do problema ($n \leq 50000$), garantindo complexidade linear $O(n)$.

A solução não apenas resolve o problema de forma ótima, mas também ilustra como estruturas de grafos e algoritmos clássicos podem ser aplicados para modelar desafios práticos, como a navegação em topologias complexas.

Características do Grafo

A ilha do enunciado descreve um grafo não orientado acíclico conexo i.e. uma árvore. Nesta árvore, cada nó representa um habitante e cada arco representa uma conexão entre os 2 habitantes de tal modo que todos os habitantes estão conectados entre si através de arestas na árvore.

Implementação do Grafo

Para implementar este grafo utilizamos listas. Para tal utilizamos uma classe “Ilha” que contém uma variável “habitantes” com o número de habitantes. Assim, cada habitante tem uma lista associada que armazena objetos Edge, que indicam os seus vizinhos diretos, fazendo com que se o habitante 1 tem conexão com o habitante 2, o 2 também tem conexão com o 1 .

Descrição do Algoritmo

Introdução:

Primeiramente processamos o input dado e criamos a árvore correspondente. Depois de termos a árvore construída, com as características previamente definidas, aplicamos 2 percursos em largura (BFS). O primeiro BFS serve para achar o nó mais distante de um nó arbitrário, neste caso o nó 1, e guardá-lo na variável noMaisDistante . No segundo BFS calculamos a distância máxima ao noMaisDistante e guardamo-lo na variável distanciaMáxima, que nos dará o diâmetro da árvore e o resultado para o problema.

1. processamento da árvore

- a. lemos do input o número de habitantes(n) e as $n-1$ conexões entre eles
- b. criamos um grafo com n elementos armazenado como uma lista de listas ($\text{List}\langle\text{List}\langle\text{Edge}\rangle\rangle$), onde cada índice corresponde a um habitante (de 1 a n)
- c. Para cada habitante, a lista associada armazena objetos Edge, que indicam os seus vizinhos diretos.
- d. A lista implementa conexões bidirecionais.

2. Primeira BFS: Encontrar um Extremo do Diâmetro

Objetivo: Identificar um dos nós mais distantes de um ponto inicial arbitrário (ex: nó 1).

- Durante a BFS, mantém registro do nó com a maior distância (noMaisDistante) e sua distância(distanciaMaxima) ao nó inicial .

3. Segunda BFS: Calcular o Diâmetro

- **Objetivo:** Encontrar o nó mais distante do extremo identificado na primeira BFS. A distância entre esses dois extremos é o diâmetro.
- Repete o processo da primeira BFS, mas inicia a partir do noMaisDistante encontrado na primeira BFS.
- Resultado Final:A distanciaMaxima obtida nesta segunda BFS corresponde ao diâmetro da árvore.

4. Retornar o resultado

Retorna, através da variável `distanciaMaxima` na segunda BFS, o resultado sendo este o diâmetro da árvore

Exemplo do Algoritmo

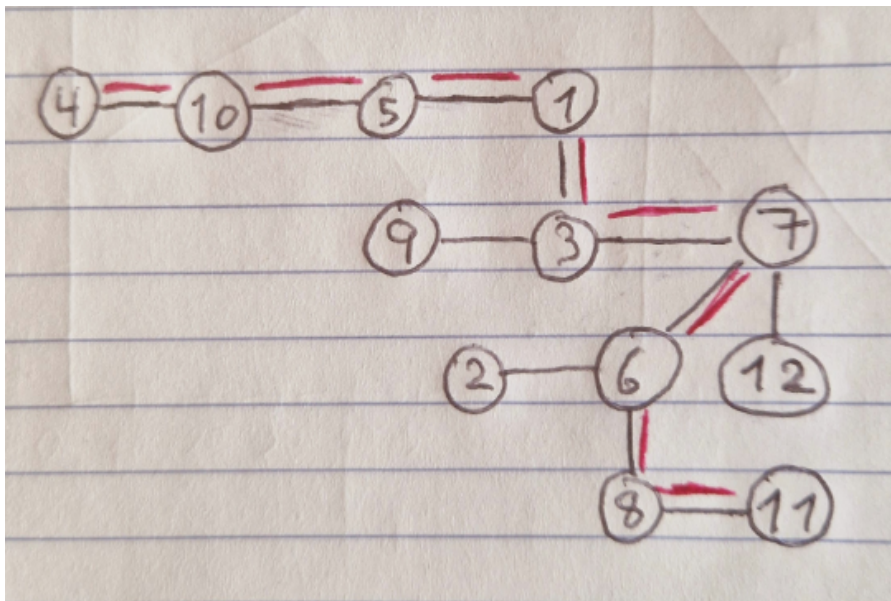


Fig. 1 - Árvore do exemplo 1 do Input do enunciado.

Este é um exemplo da árvore gerada pelo input do exemplo 1. Sendo as arestas a preto as conexões, e as arestas a vermelho o caminho mais comprido/diâmetro da árvore.

Passo a passo do processamento desta arvore :

→ Primeira BFS (Iniciando no Nó 1)

- Inicialização:

Todos os nós são marcados como white, exceto o nó 1 (grey, $d = 0$).

- Processamento:

1. Nó 1:

Vizinhos: 5 e 3.

Atualizações:

5: $d = 1$, cor = grey.

3: $d = 1$, cor = grey.

1: cor = black

Fila atualizada: [5, 3]

2. Nó 5:

Vizinhos: 1 (já visitado) e 10.

Atualizações:

10: $d = 2$, cor = grey.

5: cor = black

Fila atualizada: [3, 10].

3. Nó 3:

Vizinhos: 1 (já visitado), 7, 9.

Atualizações:

7: $d = 2$, cor = grey.

9: $d = 2$, cor = grey.

3: cor = black

Fila atualizada: [10, 7, 9].



4. Nó 10:

Vizinhos: 5 (já visitado) e 4.

Atualizações:

4: $d = 3$, cor = grey.

10: cor = black

Fila atualizada: [7, 9, 4].

5. Nó 7:

Vizinhos: 3 (já visitado), 12, 6.

Atualizações:

12: $d = 3$, cor = grey.

6: $d = 3$, cor = grey.

7: cor = black

Fila atualizada: [9, 4, 12, 6].

6. Nó 9:

Vizinhos: 3 (Sem vizinhos não visitados)

Atualizações:

9: cor = black

Fila atualizada: [4, 12, 6].

7. Nó 4

Vizinhos: 10 (Sem vizinhos não visitados).

Atualizações:

4: cor = black

Fila atualizada: [12, 6].

8. Nó 12:

Vizinhos: 7(Sem vizinhos não visitados).

Atualizações:

12: cor = black

Fila atualizada: [6]

9. Nó 6:

Vizinhos: 7 (já visitado), 2, 8.

Atualizações:

2: d = 4, cor = grey.

8: d = 4, cor = grey.

6: cor = black

Fila atualizada: [2, 8].

10. Nó 2:

Vizinhos: 6(Sem vizinhos não visitados).

Atualizações:

2: cor = black

Fila atualizada: [8].

11.Nó 8:

Vizinhos: 6 (já visitado) e 11.

Atualizações:

11: d = 5, cor = grey.

8: cor = black

Fila atualizada: [11].



12. Nó 11:

Vizinhos: 8 (Sem vizinhos não visitados).

Atualizações:

11: cor = black.

Fila atualizada: [].

Resultado da Primeira BFS:

Nó mais distante: 11 (distância = 5).

→ Segunda BFS (Iniciando no Nó 11)

- Inicialização:

Todos os nós levam reset para white, exceto o nó 11 (grey, $d = 0$).

- Processamento:

1. Nó 11:

Vizinho: 8.

Atualizações:

8: $d = 1$, cor = grey.

11: cor = black

Fila atualizada: [8].

2. Nó 8:

Vizinhos: 11 (já visitado), 6.

Atualizações:

6: $d = 2$, cor = grey.

8: cor = black

Fila atualizada: [6].



3. Nó 6:

Vizinhos: 8 (já visitado), 7, 2.

Atualizações:

7: $d = 3$, cor = grey.

2: $d = 3$, cor = grey.

6: cor = black

Fila atualizada: [7, 2].

4. Nó 7:

Vizinhos: 6 (já visitado), 3, 12.

Atualizações:

3: $d = 4$, cor = grey.

12: $d = 4$, cor = grey.

7: cor = black

Fila atualizada: [2, 3, 12].

5. Nó 2:

Vizinhos: 6 (Sem vizinhos não visitados).

Atualizações:

2: cor = black

Fila atualizada: [3, 12].

6. Nó 3:

Vizinhos: 7 (já visitado), 1, 9.

Atualizações:

1: $d = 5$, cor = grey.

9: $d = 5$, cor = grey.

3: cor = black

Fila atualizada: [12, 1, 9].

7. Nó 12:

Vizinhos: 7 (Sem vizinhos não visitados).

Atualizações:

12: cor = black

Fila atualizada: [1, 9].

8. Nó 1:

Vizinhos: 3 (já visitado), 5.

Atualizações:

5: d = 6, cor = grey.

1: cor = black

Fila atualizada: [9, 5].

9. Nó 9:

Vizinhos: 3 (Sem vizinhos não visitados).

Atualizações:

9: cor = black

Fila atualizada: [5].

10. Nó 5:

Vizinhos: 1 (já visitado), 10.

Atualizações:

10: d = 7, cor = grey.

5: cor = black

Fila atualizada: [10].



11.Nó 10:

Vizinhos: 5 (Sem vizinhos não visitados).

Atualizações:

4: $d = 8$, cor = grey.

10: cor = black

Fila atualizada: [4].

12. Nó 4:

Vizinhos: 10 (Sem vizinhos não visitados).

Atualizações:

4: cor = black

Fila atualizada: [].

Resultado da Segunda BFS:

Nó mais distante: 4 (distância = 8).

→ Saída do Resultado

Diâmetro da árvore: 8 (caminho: $4 \rightarrow 10 \rightarrow 5 \rightarrow 1 \rightarrow 3 \rightarrow 7 \rightarrow 6 \rightarrow 8 \rightarrow 11$).

Complexidade Temporal

1. Execução do Algoritmo BFS (Procura em Largura)

O BFS é executado duas vezes para calcular o diâmetro da árvore.

Cada BFS visita todos os “V” nós e “E” arestas exatamente uma vez, resultando na complexidade de $O(V+E)$ por execução.

Ou seja $O(V+E) + O(V+E) = 2 O(V+E)$.

2. Total

$BFS(O(V+E)) = O(V+E)$

Complexidade Espacial

1. Lista de Adjacências

Armazena “V” listas (uma por habitante), cada uma contendo as arestas conectadas.

Como há “E” arestas no total, o espaço ocupado é $O(V + E)$.

2. Array de Vértices

Um array de tamanho V (índices de 1 a n) armazena os estados dos nós.

Isso consome $\Theta(V)$ espaço.

3. Fila do BFS

No pior caso (Árvore Linear), a fila armazena até “V” nós simultaneamente, resultando em $O(V)$

4. Total

Lista de adjacências($O(V+E)$) + Array de Vértices($\Theta(V)$) +
+Fila($O(V)$) = $O(V+E)$